

Student Enrollment System

Technical Deep-Dive, Code Quotes, Integrity Defenses, and Verbatim Interview Responses

Role: Senior Full-Stack TypeScript Engineer

Stack: Fastify · Drizzle ORM · PostgreSQL (Local, No Docker) · React · TanStack Query · Zod

Test Suite: 14/14 Real PostgreSQL Tests Passing

Table of Contents

1. Project Overview & Philosophy

2. Folder & Layering Architecture

3. Database Schema & Integrity

4. Endpoints Deep Dive & Zero N+1

5. Automated Test Suite Breakdown
6. Frontend Architecture & State

7. Consolidated Design Decisions

8. Anticipated Interview Q&A (18 Scenarios)

9. Quick Reference Cheat Sheet

1. Project Overview & Architectural Philosophy

The **Student Enrollment System** is a full-stack, production-grade application engineered as a take-home technical deliverable for **Tissoura.ai**. It provides academic administrators with a unified dashboard and a hardened REST API to register students, define unique course subjects, enroll students across academic years with letter grades, and perform multi-dimensional filtered searches.

Stack Selection & Justification

- Fastify (v5) + TypeScript:** Fastify was mandated for its ultra-low routing overhead (~2-3x throughput of Express), native JSON schema encapsulation, and lifecycle hooks. Paired with strict TypeScript, it guarantees compile-time type safety across the entire request-response cycle.
- Drizzle ORM (v0.39):** Unlike bulky ActiveRecord ORMs (Prisma, TypeORM), Drizzle functions as a zero-overhead TypeScript SQL dialect. Queries compile directly to transparent SQL without hidden engine runtimes, guaranteeing that complex joined queries execute without unwanted subqueries.
- PostgreSQL (v18, Local, No Docker):** Provides enterprise ACID guarantees. Used directly via native node-postgres connection pooling. Relies on engine-level constraints (foreign keys, composite primary keys) to enforce data integrity and defend against concurrency anomalies.
- React + TanStack Query (v5):** Declarative client-state management with key-based cache synchronization. Automatically invalidates queries on mutations without manual global state orchestration.
- Zod (v3.24):** Validates payloads and query parameters at the HTTP boundary, inferring immutable TypeScript types directly into handlers and services.

Architecture Philosophy: Lightweight Modular Layering (Not DDD)

The system deliberately avoids Domain-Driven Design (DDD) overhead—such as Aggregate Roots, Value Object wrappers, Domain Events, and Abstract Repository Interfaces—which introduce excessive indirection for CRUD-oriented domains. Instead, it adopts a **flat, modular layered architecture** loosely inspired by NestJS conventions: routes handle wiring, handlers parse HTTP, services execute domain logic and SQL, DTOs govern schemas, helpers house pure utilities, and shared layers centralize error handling.

2. Folder & File Structure — Explained

```
student-enrollment/
├── src/
│   ├── app.ts           # Fastify application factory (CORS, error handler, routes)
│   ├── server.ts        # Server entrypoint with port listening & graceful shutdown
│   ├── db/
│   │   ├── client.ts    # pg.Pool and Drizzle client instance + test client factory
│   │   └── schema.ts    # Drizzle schemas (students, subjects, enrollments) + relations
```

```

├── seed.ts                # Seeding script for demo data
├── inspect.ts             # Rapid terminal inspection tool
├── migrations/            # Generated SQL migrations (0000_overjoyed_micromacro.sql)
├── modules/
│   ├── students/         # Students domain module
│   │   ├── dto/students.dto.ts
│   │   ├── handlers/students.handlers.ts
│   │   ├── helpers/students.helpers.ts
│   │   ├── routes/students.routes.ts
│   │   └── services/students.service.ts
│   ├── enrollments/     # Enrollments domain module
│   │   ├── dto/enrollments.dto.ts
│   │   ├── handlers/enrollments.handlers.ts
│   │   ├── helpers/enrollments.helpers.ts
│   │   ├── routes/enrollments.routes.ts
│   │   └── services/enrollments.service.ts
│   └── subjects/        # Subjects lookup domain module
│       ├── dto/subjects.dto.ts
│       ├── handlers/subjects.handlers.ts
│       ├── routes/subjects.routes.ts
│       └── services/subjects.service.ts
├── shared/
│   ├── errors.ts         # AppError hierarchy (NotFoundError, ConflictError, etc.)
│   ├── error-handler.ts  # Centralized Fastify error mapping (AppError/Zod → HTTP)
│   └── pagination.ts     # Pagination schemas and math helpers
├── tests/
│   ├── setup.ts          # Test suite setup & TRUNCATE CASCADE beforeEach hook
│   ├── students.test.ts  # 8 integration tests against real PostgreSQL
│   └── enrollments.test.ts # 6 integration tests against real PostgreSQL
└── frontend/             # React 18 + Vite + TanStack Query SPA

```

Layer Boundaries & Separation of Concerns

- **routes/**: Fastify plugins responsible purely for URI path mapping, HTTP method binding, and wiring handlers. No request parsing or business logic exists here.
- **handlers/**: The thin HTTP transport boundary. Receives Fastify request and reply objects, validates payloads via Zod, invokes the domain service, and sets response HTTP status codes (200, 201, 204). Strictly prohibited from issuing database queries.
- **services/**: The core domain and database access layer. The *only* layer allowed to interact with Drizzle ORM. Enforces business rules and throws typed `AppError` subclasses when invariants fail.
- **dto/**: Declares Zod validation schemas and derives static TypeScript types (`z.infer`) for bodies, queries, and parameters.
- **helpers/**: Pure, stateless utility functions (e.g. grade threshold calculation, academic year range checks). No database or external side effects.
- **shared/**: Framework-wide cross-cutting concerns: error classes, centralized error dispatching, and pagination metadata builders.

Key Interview Files (Top 6):

1. `src/db/schema.ts` : Declares database columns, composite primary key, and relational joins.
2. `src/modules/enrollments/services/enrollments.service.ts` : Implements concurrency-safe insert catching PostgreSQL code 23505.
3. `src/modules/students/services/students.service.ts` : Implements single joined query filtering and zero N+1 relational fetches.
4. `src/shared/error-handler.ts` : Centralizes exception translation into RFC-compliant JSON responses.
5. `src/shared/errors.ts` : Encapsulates domain error semantics with dedicated HTTP status codes.
6. `src/app.ts` : Assembles Fastify plugins, CORS, global error handling, and routing in an isolated factory.

3. Database Schema & Integrity Guarantees

The schema is declared in `src/db/schema.ts` using Drizzle ORM's PostgreSQL core primitives:

```
export const students = pgTable("students", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const subjects = pgTable("subjects", {
  id: uuid("id").defaultRandom().primaryKey(),
  name: text("name").notNull().unique(),
});

export const enrollments = pgTable(
  "enrollments",
  {
    studentId: uuid("student_id").notNull().references(() => students.id, { onDelete: "cascade" }),
    subjectId: uuid("subject_id").notNull().references(() => subjects.id, { onDelete: "restrict" }),
    year: integer("year").notNull(),
    grade: varchar("grade", { length: 2 }).notNull(),
    createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  },
  (table) => [
    primaryKey({ columns: [table.studentId, table.subjectId, table.year] }),
  ]
);
```

Critical Architectural Rules in the Schema:

1. Composite Primary Key (`student_id, subject_id, year`):

Enforces at the database storage engine level that a student can only have *one* enrollment record per subject in a given academic year.

Why is year part of the key? Academic curricula allow a student who failed or withdrew in 2023 to legitimately retake the subject in 2024.

Why is grade NOT part of the key? A grade is a mutable attribute of an enrollment. If grade were in the primary key, a student could have multiple conflicting grades (e.g. an 'A' and an 'F') for the identical subject in the identical year.

2. Foreign Key `onDelete` Rules:

- `studentId` → `onDelete: 'cascade'`: Enrollments represent personal academic history. When a student record is deleted, their associated enrollment rows are deleted atomically via cascade, preventing orphan rows.
- `subjectId` → `onDelete: 'restrict'`: Course catalog subjects must be protected against accidental deletion if students have enrolled in them.

3. UUIDs vs. Auto-Increment Integers:

UUIDs (`defaultRandom()`) eliminate enumeration vulnerabilities, prevent competitors from inferring total system volume, and permit client-side or distributed ID generation without coordination.

4. Endpoints — Explained One by One

1. GET /students

- Request Query:** `page` (int, default 1), `limit` (int, default 10), `subject` (optional string), `minGrade` (optional: 'A'|'B'|'C'|'D'|'F'). Validated by `studentFilterQuerySchema`.
- Success Response:** `200 OK` with `{ data: Student[], meta: PaginationMeta }`.
- Error Cases:** `400 ValidationError` if page/limit are negative or minGrade is invalid.
- Exact Service Implementation (Single Joined Query):**

```
// 1. Total count of distinct students matching filter
const [countResult] = await this.db
  .select({ total: sql<number>`count(distinct ${students.id})::int` })
  .from(students)
```

```

    .innerJoin(enrollments, eq(students.id, enrollments.studentId))
    .innerJoin(subjects, eq(enrollments.subjectId, subjects.id))
    .where(and(...filterConditions));

// 2. Single joined select with distinct on student id
const rows = await this.db
    .selectDistinctOn([students.id], { id: students.id, name: students.name, createdAt: students.createdAt })
    .from(students)
    .innerJoin(enrollments, eq(students.id, enrollments.studentId))
    .innerJoin(subjects, eq(enrollments.subjectId, subjects.id))
    .where(and(...filterConditions))
    .orderBy(students.id)
    .limit(limit).offset(offset);

```

- **Why Zero N+1:** A student with 5 qualifying enrollments is resolved in a single `INNER JOIN` with `selectDistinctOn`, producing an exact single student row without separate child queries.

2. POST /students

- **Request Body:** `{ "name": "Alice Johnson" }` (min 1 char, trimmed). Schema: `createStudentSchema`.
- **Success Response:** `201 Created` returning `{ id, name, createdAt }`.
- **Error Cases:** `400 ValidationError` if name is empty, missing, or whitespace.

3. DELETE /students/:id

- **Request Params:** `id` (UUID). Schema: `studentIdParamSchema`.
- **Success Response:** `204 No Content`.
- **Error Cases:** `404 NotFoundError` if student ID does not exist; `400 ValidationError` if ID is not a valid UUID.
- **Cascade Action:** Database triggers cascade deletion on `enrollments`.

4. POST /students/:id/enroll (Concurrency & Race-Condition Safe)

- **Request:** Param `id` (UUID). Body: `{ subjectId: UUID, year: int (1990-2100), grade: 'A'|'B'|'C'|'D'|'F' }`.
- **Success Response:** `201 Created` returning `{ studentId, subjectId, year, grade, createdAt }`.
- **Error Cases:** `404 NotFoundError` (missing student or subject), `400 ValidationError` (invalid grade/year), `409 ConflictError` (duplicate enrollment).
- **Exact Service Implementation:**

```
// 1. Check student and subject existence
const student = await this.db.query.students.findFirst({ where: eq(students.id, studentId) });
if (!student) throw new NotFoundError(`Student with ID '${studentId}' not found`);

const subject = await this.db.query.subjects.findFirst({ where: eq(subjects.id, data.subjectId) });
if (!subject) throw new NotFoundError(`Subject with ID '${data.subjectId}' not found`);

// 2. Attempt insert and catch Postgres unique violation error code 23505
try {
  const [newEnrollment] = await this.db.insert(enrollments).values({
    studentId, subjectId: data.subjectId, year: data.year, grade: data.grade,
  }).returning();
  return newEnrollment;
} catch (error: any) {
  if (error?.code === "23505") {
    throw new ConflictError(`Student is already enrolled in ${subject.name} for year ${data.year}`);
  }
  throw error;
}
```

- **Concurrency Defense:** Rather than relying on an inherently racy check-then-insert pattern (where two concurrent requests can both pass existence checks before either inserts), we rely on PostgreSQL's composite primary key index. Concurrent duplicate inserts trigger code `23505`, mapped cleanly to `409 Conflict`.

5. GET /students/:id (Zero N+1 Relational Loading)

- **Request:** Param `id` (UUID).
- **Success Response:** `200 OK` with student metadata and nested `enrollments` containing joined `subjectName`.
- **Error Cases:** `404 NotFoundError` if student does not exist.
- **Exact Service Implementation:**

```
const student = await this.db.query.students.findFirst({
  where: eq(students.id, id),
  with: {
    enrollments: {
      with: { subject: true },
      orderBy: [desc(enrollments.year)],
    },
  },
});
if (!student) throw new NotFoundError(`Student with ID '${id}' not found`);
```

- **Why Zero N+1:** Drizzle's relational query compiles into a single SQL statement that joins the subjects table directly on `enrollments.subject_id = subjects.id`. It never issues a secondary query per enrollment.

5. Automated Test Suite — 14 Real Postgres Tests

Tests are located in `tests/students.test.ts` and `tests/enrollments.test.ts`, executed via Vitest against `student_enrollment_test` using Fastify's native `app.inject()`.

File	Test Description	Asserted Status	Failure Guarded Against
<code>students.test.ts</code>	Happy path student creation	<code>201 Created</code>	Ensures valid records persist and return generated UUID and timestamp.

File	Test Description	Asserted Status	Failure Guarded Against
<code>students.test.ts</code>	Validation on empty or missing name	400 Bad Request	Prevents empty or whitespace-only students from polluting the database.
<code>students.test.ts</code>	Filter by subject & minGrade (joined)	200 OK	Verifies single joined query correctly filters students meeting grade threshold.
<code>students.test.ts</code>	Filter with no matching records	200 OK (empty)	Guarantees graceful response when filters match zero records.
<code>students.test.ts</code>	Single query student + enrollments + subject	200 OK	Verifies zero N+1 relational fetching of nested enrollments.
<code>students.test.ts</code>	Get non-existent student ID	404 Not Found	Guarantees missing IDs return RFC-compliant 404 rather than empty objects.
<code>students.test.ts</code>	Delete student & cascade enrollments	204 No Content	Verifies cascade deletion clears child enrollments without foreign key failure.
<code>students.test.ts</code>	Delete non-existent student ID	404 Not Found	Prevents silent no-ops when deleting nonexistent resources.
<code>enrollments.test.ts</code>	Enroll student happy path	201 Created	Verifies valid enrollment persistence and returned payload.
<code>enrollments.test.ts</code>	Enroll with non-existent student	404 Not Found	Guarantees foreign key references are validated before insertion.
<code>enrollments.test.ts</code>	Enroll with non-existent subject	404 Not Found	Prevents orphan enrollments against non-existent subjects.
<code>enrollments.test.ts</code>	Validation on invalid grade or year	400 Bad Request	Rejects invalid grades (e.g. 'Z') and unrealistic years (e.g. 1800).
<code>enrollments.test.ts</code>	Sequential duplicate enrollment	409 Conflict	Ensures duplicate <code>(studentId, subjectId, year)</code> returns 409 Conflict.
<code>enrollments.test.ts</code>	Concurrent enrollment (Promise.all race)	[201, 409]	Simultaneously fires two identical requests; verifies exactly 1 succeeds and 1 conflicts.

The Concurrency Test Explained Step-by-Step

```
const [res1, res2] = await Promise.all([
  app.inject({ method: "POST", url: `/students/${student.id}/enroll`, payload }),
  app.inject({ method: "POST", url: `/students/${student.id}/enroll`, payload }),
]);
const statusCodes = [res1.statusCode, res2.statusCode].sort();
expect(statusCodes).toEqual([201, 409]);
```

What happens step-by-step: Both requests enter Fastify concurrently. Both successfully pass the initial `findFirst` checks for student and subject existence. Both reach the `db.insert(enrollments)` statement nearly simultaneously. PostgreSQL executes transaction isolation on the primary key B-Tree index: the first insert acquires the lock and succeeds (status `201`); the second insert encounters a primary key violation, raising PostgreSQL error `23505`. The service's catch block intercepts `23505`, throws `ConflictError`, and Fastify replies with status `409`. Exactly one row exists in the database.

Test Isolation via `vitest.config.ts` & `setup.ts`

Test isolation is achieved by configuring `test.env` in `vitest.config.ts`: `DATABASE_URL`: `process.env.DATABASE_TEST_URL || "postgresql://postgres@localhost:5432/student_enrollment_test"`. This injects the test database URL into the worker process *before* any ESM modules are linked. Inside `tests/setup.ts`, a `beforeEach` hook executes: `TRUNCATE TABLE "enrollments", "students", "subjects" CASCADE;`, guaranteeing complete test hermeticism in ~2 seconds.

6. Frontend Architecture & TanStack Query State

The frontend is an SPA built with **React 18 + Vite + TanStack Query (v5)**. It prioritizes clean UX, responsive styling without heavy CSS frameworks, and reactive cache invalidation.

Components & Responsibilities

- StudentsListPage** (`pages/StudentsListPage.tsx`): Main dashboard view. Coordinates filter states, search inputs, pagination controls, and modal toggles.
- StudentDetailPage** (`pages/StudentDetailPage.tsx`): Student profile view. Displays student metadata and an enrolled subjects table showing academic year, grade badge, and enrollment date.
- FilterBar** (`components/FilterBar.tsx`): Filter controls with dynamic subject dropdown (loaded via `useSubjects`) and minimum grade threshold selector.
- StudentsTable** (`components/StudentsTable.tsx`): Displays paginated student rows with quick actions ("View Details", "+ Enroll", "Delete").
- AddStudentModal** (`components/AddStudentModal.tsx`): Modal form to create a new student with live error banners.
- EnrollModal** (`components/EnrollModal.tsx`): Modal form to enroll a student. Dynamically maps HTTP 409 conflict errors to inline warning banners.
- Badge** (`components/Badge.tsx`): Renders color-coded grade pills (Grade A: Emerald, Grade B: Blue, Grade C: Amber, Grade D: Orange, Grade F: Red).
- Pagination** (`components/Pagination.tsx`): Displays current page, total records, and previous/next page navigation buttons.

TanStack Query Cache Key Strategy & Mutations

Hook	Query Key	Behavior & Invalidation
<code>useStudents(filters)</code>	<code>['students', { page, limit, subject, minGrade }]</code>	Refetches automatically whenever page, subject, or minGrade changes. Stale time: 30s.
<code>useStudent(id)</code>	<code>['student', id]</code>	Queries single student details with nested enrollments. Enabled only when ID is present.
<code>useSubjects()</code>	<code>['subjects']</code>	Caches academic subjects for dropdowns with 5-minute stale time.
<code>useCreateStudent()</code>	Mutation	On success: calls <code>queryClient.invalidateQueries({ queryKey: ['students'] })</code> .
<code>useDeleteStudent()</code>	Mutation	On success: invalidates <code>['students']</code> , updating the list immediately.
<code>useEnrollStudent(id)</code>	Mutation	On success: invalidates both <code>['student', id]</code> and <code>['students']</code> .

7. Consolidated Design Decisions & Trade-offs

- 1. Cascade vs. Restrict on Delete:**

Enrollments belong strictly to a student's personal educational record. When a student is removed, cascading deletion clears child enrollments atomically. However, subjects belong to the school catalog; deleting a subject is restricted if students are actively enrolled in it to prevent corrupting academic transcripts.
- 2. Single Joined Queries vs. Relational Zero N+1:**

 - For `GET /students/:id`: Drizzle's relational API generates a single SQL query with joins, mapping nested subjects into memory in a single trip.
 - For `GET /students?subject=...&minGrade=...`: We utilize an explicit `INNER JOIN` across students, enrollments, and subjects with `selectDistinctOn` and a single distinct count query. This prevents issuing N queries for N students.

3. Race-Condition Defense via Engine Constraints:

Application-level "check-then-insert" checks are fundamentally racy under concurrent requests. We enforce a DB-level composite primary key `(student_id, subject_id, year)` and catch Postgres unique violation error code `23505` in the service layer, translating it into HTTP 409 Conflict.

What I would do differently with more time (Interview Self-Awareness):

- **Soft Deletes:** In a real production university system, students and enrollments are rarely hard-deleted. Adding a `deleted_at` timestamp column with Drizzle filters would support regulatory auditing.
- **OpenAPI / Swagger Generation:** Integrate `@fastify/swagger` and `zod-to-json-schema` to automatically generate interactive Swagger UI documentation directly from our Zod DTOs.
- **Authentication & Role-Based Access Control (RBAC):** Add Fastify plugins for JWT or session authentication, distinguishing "Admin" (create/delete) from "Student" (view only).

8. Anticipated Interview Questions & Model Answers

Q1: "Walk me through what happens when I hit POST /students/:id/enroll twice at the same time."

Both HTTP requests hit Fastify in parallel and pass Zod payload validation. Handlers call `enrollmentsService.enrollStudent`. Both requests check whether the student and subject exist in PostgreSQL—both checks return true. Next, both requests execute `INSERT INTO enrollments` simultaneously. PostgreSQL serializes row insertion on the composite primary key index `(student_id, subject_id, year)`. One request acquires the write lock first and inserts successfully, returning status 201. The second request encounters a unique constraint violation and PostgreSQL raises error code `23505`. Our service catch block inspects `error.code === '23505'`, wraps it into a domain `ConflictError`, and Fastify's centralized error handler transforms it into an HTTP 409 Conflict response.

"I intentionally avoided check-then-insert because it's vulnerable to race conditions under load. Instead, I rely on the database composite primary key constraint and explicitly catch Postgres error code 23505 in the service layer."

Q2: "Why is grade not part of the composite primary key?"

Primary keys must uniquely identify the entity—which in this domain is the student's enrollment in a subject for an academic year. The grade is a mutable attribute of that enrollment, not an identifier. If grade were part of the primary key, the composite key would be `(studentId, subjectId, year, grade)`. That would allow a student to simultaneously have an enrollment with grade 'A' and another with grade 'F' for the exact same subject in the exact same year.

"Grade is a property of the enrollment, not its identity. If grade were in the key, a student could hold multiple contradictory grades for the same course in the same year."

Q3: "How do you know GET /students/:id doesn't do N+1 queries — show me."

In `src/modules/students/services/students.service.ts`, we use Drizzle's relational query API: `db.query.students.findFirst({ where: eq(students.id, id), with: { enrollments: { with: { subject: true } } } })`. Drizzle translates this into an optimized SQL query using joins and json aggregation in a single round-trip. At no point is there a `for` loop or a secondary `db.select().from(subjects).where(...)` per enrollment.

"I use Drizzle's relational findFirst with nested relations. It compiles into a single SQL statement that fetches the student, enrollments, and subject names in one round-trip."

Q4: "What would you change if this needed to support 10,000 students?"

For 10,000 students, the current PostgreSQL schema and query architecture easily handle the load because queries use index scans on UUID PKs and composite indexes. However, I would: (1) Add a composite B-Tree index on `(subject_id, grade)` to optimize the joined filter query; (2) Implement cursor-based keyset pagination (e.g. `WHERE id > :lastId`) instead of `OFFSET` pagination, which degrades at high page depths; (3) Add connection pool scaling in `pg.Pool`; (4) Add Redis caching for frequently accessed subject catalogs.

"The composite primary key already indexes the join. For 10,000 students, I would add a composite index on (subject_id, grade) and switch from OFFSET pagination to keyset cursor pagination."

Q5: "Why UUID instead of auto-increment integers?"

UUID v4 (`defaultRandom()`) avoids predictable ID enumeration attacks (where an attacker iterates `/students/1`, `/students/2`). It prevents external parties from guessing total database volume. It also allows IDs to be safely generated by client distributed services or seed scripts without requiring round-trip database sequence locks.

"UUIDs prevent sequential enumeration attacks, protect business metrics from being guessed, and support distributed generation."

Q6: "Why did you separate handlers from services instead of putting logic in the route directly?"

Separation of concerns. Routes only know about Fastify URL endpoints. Handlers act as the HTTP transport layer: they parse headers, validate Zod schemas, invoke services, and format status codes. Services contain pure business rules and database queries without any concept of Fastify `request` or `reply` objects. This makes services directly unit-testable and reusable across background queues, CLI scripts, or alternative transports (e.g. gRPC or WebSockets) without mocking HTTP contexts.

"Handlers manage HTTP transport and validation; services manage business logic and SQL. This decouples database operations from Fastify, making services reusable and testable without HTTP mocks."

Q7: "What happens if Zod validation fails — walk me through the full error path."

When a request hits a handler (e.g. `createStudentHandler`), it calls `createStudentSchema.parse(request.body)`. If validation fails, Zod throws a `ZodError` containing an array of validation issues. The handler does not catch this error; it bubbles up to Fastify's centralized `setErrorHandler(errorHandler)` in `src/shared/error-handler.ts`. The error handler recognizes `error instanceof ZodError`, formats the path and issue message, and replies with an HTTP 400 Bad Request and structured JSON: `{ statusCode: 400, error: 'ValidationError', message: 'Request validation failed', issues: [...] }`.

"Zod throws a `ZodError`, which bubbles to Fastify's centralized `errorHandler`. It maps the error to a clean 400 Bad Request with field-level issues."

Q8: "How would you add role-based access control (RBAC) without a rewrite?"

I would implement a Fastify `preHandler` hook or plugin (e.g. `verifyRole(['ADMIN'])`). Fastify's plugin architecture allows scoping hooks to specific route prefixes or passing them as `preHandler` options in route definitions: `fastify.delete('/students/:id', { preHandler: verifyRole(['ADMIN']) }, studentsHandler.deleteStudent)`. Because our handlers and services don't handle auth concerns, zero business logic needs modification.

"I would use Fastify `preHandler` hooks on route definitions. Because handlers and services are cleanly decoupled, no service logic needs to change."

Q9: "Why cascade on student delete but restrict on subject delete?"

An enrollment represents a student's personal participation record. If a student is purged from the database, leaving orphan enrollment records breaks referential integrity. In contrast, academic subjects belong to the institutional catalog. Deleting a subject while students have historical grades in it would destroy legitimate transcript records. Therefore, subject deletion must be restricted if active enrollments exist.

"Enrollments are lifecycle-bound to students, so deleting a student cascades. Subjects are institutional catalog items, so deleting them is restricted if student records reference them."

Q10: "How does the grade-ranking helper work in SQL without custom PostgreSQL enums?"

In `src/modules/students/helpers/students.helpers.ts`, the `getGradesMeetingMinimum(minGrade)` pure helper maps letter grades to rank orders (A:5, B:4, C:3, D:2, F:1). When a user requests `minGrade=B`, the helper returns `['A', 'B']`. The service then queries PostgreSQL using Drizzle's `inArray(enrollments.grade, ['A', 'B'])`. This translates to standard SQL `IN ('A', 'B')`, which is simple, fully indexable, and avoids complex PostgreSQL `ENUM` cast overhead.

"The helper translates `minGrade 'B'` into an array `['A', 'B']`, and Drizzle executes a standard SQL `IN ('A', 'B')` query, which is index-friendly."

Q11: "What happens if a student has multiple enrollments matching the filter — why don't they appear twice?"

If a student has grade 'A' in Math 2023 and grade 'A' in Math 2024, a naive join returns two rows for that student. We prevent duplicates in two places: (1) In the count query, we execute `count(distinct students.id)`; (2) In the data query, we use PostgreSQL's native `selectDistinctOn([students.id])` ordered by `students.id`. This guarantees that each matching student appears exactly once in the paginated results.

"I used PostgreSQL `selectDistinctOn([students.id])` and `count(distinct students.id)` in the joined query so each matching student only appears once."

Q12: "Why did you use Fastify instead of Express or NestJS?"

Express is minimalist but lacks modern async ergonomics and has higher routing overhead. NestJS provides good structure but introduces heavy abstractions (decorators, metadata reflection, DI containers) that constitute over-engineering for a focused take-home. Fastify hits the exact sweet spot: modern async/await first, 2-3x faster than Express, built-in schema compilation, and clean encapsulation via plugins without runtime magic.

"Fastify gives the performance and schema-driven rigor of modern TypeScript without NestJS's heavy dependency-injection boilerplate."

Q13: "Why Drizzle ORM instead of Prisma or TypeORM?"

Prisma relies on a separate Rust binary engine with overhead, generates large schema files, and makes complex joins or distinct counts clunky. TypeORM relies heavily on experimental decorators and has known maintenance issues. Drizzle ORM is pure TypeScript, matches SQL directly, produces zero hidden queries, and allows full relational loading (zero N+1) alongside raw SQL escape hatches.

"Drizzle gives SQL-like transparency with full TypeScript type safety, zero runtime engine bloat, and predictable query performance."

Q14: "How did you solve the ESM import hoisting issue in Vitest during testing?"

In ECMAScript Modules (ESM), `import` statements are hoisted and evaluated before top-level runtime code. Setting `process.env.DATABASE_URL = ...` inside a setup file happens after imported modules (like the database client) have already evaluated. I solved this by configuring Vitest's native `test.env` in `vitest.config.ts`. Vitest injects the test database URL into the worker process environment before any source files or setup files are loaded.

"In ESM, imports are hoisted before runtime statements. I configured test.env in vitest.config.ts so the test environment variable is set before any module loads."

Q15: "How does TanStack Query keep the UI in sync without Redux or Zustand?"

Server state is fundamentally different from client UI state. TanStack Query treats the backend as the single source of truth. By keying queries with `['students', filters]`, any filter change triggers a background fetch. When mutations run (like `useEnrollStudent`), the `onSuccess` handler calls `queryClient.invalidateQueries({ queryKey: ['student', studentId] })`. TanStack Query marks the cached data as stale and refetches immediately, keeping the UI synchronized with zero manual state duplication.

"TanStack Query manages server cache rather than client state. Mutations invalidate query keys on success, triggering automatic background refetches."

Q16: "Why use TRUNCATE CASCADE in beforeEach instead of transactional rollbacks?"

While transaction rollback is fast, it requires sharing a single transactional database client across the test runner and the Fastify app, which complicates testing endpoints that execute their own transactions or connection pool operations. Fastify integration tests run against real HTTP endpoints via `app.inject()`. Using `TRUNCATE TABLE ... CASCADE` resets real database tables in single-digit milliseconds without coupling Fastify's pool to test transactions.

"TRUNCATE CASCADE tests the exact connection pooling Fastify uses in production, resetting state in milliseconds without mocking transactions."

Q17: "How would you handle database migrations in a zero-downtime production deployment?"

I would follow the Expand/Contract migration pattern. For schema changes: (1) Expand phase: apply additive migrations (e.g. adding nullable columns or new tables) using `drizzle-kit migrate` before deploying new backend code; (2) Deploy the updated backend code that utilizes the new schema; (3) Contract phase: once the new backend is healthy, remove deprecated columns or constraints in a subsequent migration.

"I use the Expand/Contract pattern: apply additive schema changes first, deploy the code, and clean up deprecated columns in a follow-up migration."

Q18: "How does the pagination logic handle edge cases like negative pages or excessive limits?"

The Zod pagination schema enforces integer validation with bounds: `page:`

`z.coerce.number().int().positive().default(1)` and `limit:`

`z.coerce.number().int().positive().max(100).default(10)`. Invalid or negative values fail validation with 400. In `src/shared/pagination.ts`, `totalPages` is computed as `Math.ceil(total / limit) || 1`, and `hasNextPage` / `hasPrevPage` boolean flags prevent clients from navigating out of bounds.

"Zod enforces positive integers and caps limit at 100 at the HTTP boundary, while our pagination helper computes safe totalPages and boolean bounds flags."

9. Quick Reference Cheat Sheet

Core Operational Commands

Action	Command	Description
Start All (Dev)	<code>pnpm dev</code>	Concurrently starts Fastify on :3000 and Vite React on :5173.
Run Tests	<code>pnpm test</code>	Executes 14 Vitest integration tests against real PostgreSQL.
Migrate DB	<code>pnpm db:migrate</code>	Applies Drizzle SQL migrations to PostgreSQL.
Seed Data	<code>pnpm db:seed</code>	Populates 5 subjects, 5 students, and 6 sample enrollments.
Terminal Inspect	<code>pnpm db:inspect</code>	Prints ASCII tables of subjects, students, and enrollments in 1s.
Drizzle Studio	<code>pnpm db:studio</code>	Opens visual web dashboard at <code>https://local.drizzle.studio</code> .

Endpoint & Status Codes Matrix

Method & Path	Success Status	Error Statuses	Key Query / Protection
<code>GET</code> <code>/students</code>	<code>200 OK</code>	<code>400</code>	Single joined query (<code>students ⋈ enrollments ⋈ subjects</code>) with <code>selectDistinctOn</code> . Zero N+1.
<code>POST</code> <code>/students</code>	<code>201 Created</code>	<code>400</code>	Validates name non-empty via Zod, inserts student, returns UUID.
<code>DELETE</code> <code>/students/:id</code>	<code>204 No Content</code>	<code>400</code> , <code>404</code>	Deletes student record; cascade deletes associated enrollments in DB.
<code>POST</code> <code>/students/:id/enroll</code>	<code>201 Created</code>	<code>400</code> , <code>404</code> , <code>409</code>	Catches Postgres code <code>23505</code> on composite PK; returns <code>409 Conflict</code> . Race-condition safe.
<code>GET</code> <code>/students/:id</code>	<code>200 OK</code>	<code>400</code> , <code>404</code>	Drizzle relational query (<code>with: { enrollments: { with: { subject } } }</code>). Zero N+1.
<code>GET</code> <code>/subjects</code>	<code>200 OK</code>	<code>500</code>	Lists all academic subjects alphabetically for frontend selectors.
<code>GET</code> <code>/health</code>	<code>200 OK</code>	—	Fastify health check returning <code>{ status: "ok", timestamp }</code> .

Error Classes & HTTP Status Mapping

Error Class / Code	HTTP Status	Trigger Condition
<code>ValidationError</code> / <code>ZodError</code>	<code>400 Bad Request</code>	Payload schema violation (empty name, invalid UUID, grade not in A-F, year < 1990).
<code>NotFoundError</code>	<code>404 Not Found</code>	Student or Subject ID does not exist in the database.
<code>ConflictError</code> / <code>23505</code>	<code>409 Conflict</code>	Duplicate enrollment for the same student, subject, and academic year.
<code>InternalServerError</code>	<code>500 Internal Error</code>	Unexpected unhandled runtime exceptions.